

// training

Zustand

0 → 1 → Production

State Management สำหรับ React/JavaScript ที่เบา เร็ว และ scale ได้จริง

DURATION

120 min

AUDIENCE

React Jr–Mid

FORMAT

Lecture + Workshop

Agenda — 120 นาที

เส้นทางจาก 0 → 1 → Production

- | | | | | | |
|----|--|--------|----|-------------------------------|--------|
| 01 | Intro & Why Zustand | 10 min | 02 | Install & Core Concept | 15 min |
| 03 | Patterns + Async | 20 min | 04 | Vanilla Store (outside React) | 10 min |
| 05 | Middleware: persist / devtools / immer | 20 min | 06 | SSR / Next.js Hydration | 10 min |
| 07 | Testing Strategy | 10 min | 08 | Workshop: Mini App | 20 min |
| 09 | Production Checklist + Q&A | 5 min | | | |

Who is this for?

Prerequisites & เป้าหมายการเรียนรู้

- React dev ระดับ junior – mid ที่เคยใช้ `useState/useEffect/useContext`
- เคยทำโปรเจกต์ React + Vite/Next.js อย่างน้อย 1 ตัว
- เข้าใจ JavaScript ES6+ (destructuring, arrow fn, `async/await`)
- ไม่จำเป็นต้องรู้ Redux มาก่อน — แต่ถ้ารู้จะเปรียบเทียบได้ดียิ่งขึ้น

เมื่อจบคอร์สนี้คุณจะ...

- ✓ สร้าง Zustand store ได้เองจากศูนย์
- ✓ ออกแบบ selector ลด re-render เป็น
- ✓ ใช้ `persist` + `devtools` + `immer` ได้คล่อง
- ✓ จัดการ SSR / hydration mismatch ได้
- ✓ เขียน unit test สำหรับ store ได้
- ✓ มี checklist ก่อนขึ้น production

01

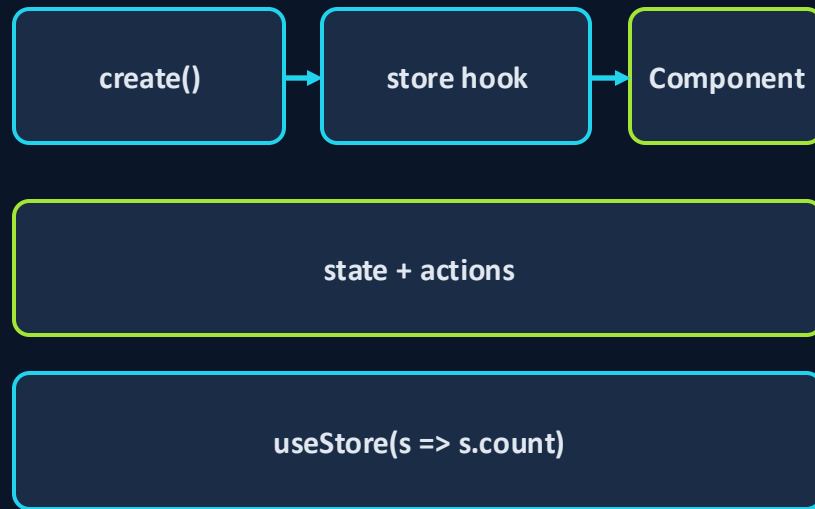
Intro

Zustand คืออะไร และทำไมต้องใช้

Zustand คืออะไร?

A small, fast, scalable state-management for React

- State management library ขนาดเล็ก (~1 KB gzipped)
- Hooks-first: ใช้ store เหมือน custom hook
- ไม่ต้อง wrap ด้วย <Provider> — store อยู่ใน module scope
- API ง่ายๆ: create, set, get, subscribe เท่านั้น
- Performance ดีด้วย selector + shallow equality



→ subscribe เฉพาะ slice ที่เลือก

⚠ Pitfalls / ข้อควรระวัง

- Zustand ไม่ใช่ของวิเศษ — ใช้ผิดวิธี (เช่น select object ทั้งก้อน) ก็ทำให้ re-render เยอะได้

ทำไมต้อง Zustand?

เปรียบเทียบกับ Context / Redux Toolkit

Feature	Context	Redux TK	Zustand
Boilerplate	น้อย	กลาง	น้อยที่สุด
Provider	ต้อง	ต้อง	ไม่ต้อง
Selector / Re-render	ทั้งต้นไม้	ดี	ดีมาก
DevTools	—	ในตัว	ผ่าน middleware
Persist	เขียนเอง	redux-persist	middleware ในตัว
ใช้ภายนอก React	ไม่ได้	ได้	ได้ (vanilla)
Bundling size	~1 KB	~12 KB	~1 KB

⚠ Pitfalls / ข้อควรระวัง

- ตารางนี้เป็นภาพรวม — เลือกเครื่องมือตามทีม/ขนาดโปรเจกต์ ไม่ใช่ตามเทรนด์

Install & Setup

ติดตั้ง Zustand ใน React project

- รองรับ React 18+ และ Next.js 13+
- ติดตั้งผ่าน npm / yarn / pnpm ได้ทุกตัว
- Zero config — import ใช้งานได้ทันที
- ขนาด ~1 KB gzipped (core)
- TypeScript รองรับเต็มรูปแบบ

```
bash

# เลือกตัวใดตัวหนึ่ง
npm install zustand
yarn add zustand
pnpm add zustand

# import ใช้งาน
import { create } from "zustand";

// (optional) middleware
import {
  persist,
  devtools,
  createJSONStorage
} from "zustand/middleware";
```

⚠ Pitfalls / ข้อควรระวัง

- อย่า import จาก zustand/vanilla ถ้าใช้ใน React component — ใช้ create จาก 'zustand' หลัก

02

Core Concept

`store = hook + state + actions`



Core: store = hook

create() return ออกมาเป็น React hook

- create(initializer) → return useStore (เป็น hook)
- initializer รับ (set, get) → คืน object ที่มี state + actions
- set: อัปเดต state แบบ shallow merge
- get: อ่าน state ปัจจุบันใน action
- เรียก useStore() ใน component เพื่อ subscribe

```
import { create } from "zustand";

export const useCounterStore =
  create((set, get) => ({
    count: 0,
    inc: () =>
      set((s) => ({ count: s.count + 1 })),
    dec: () =>
      set((s) => ({ count: s.count - 1 })),
    reset: () => set({ count: 0 }),
  }));

// ใน component
const count = useCounterStore((s) => s.count);
const inc = useCounterStore((s) => s.inc);
```

set & get ใน action

เข้าถึง state ปัจจุบันและอัปเดตอย่างปลอดภัย

- set รับ partial หรือ updater fn
- get() ใช้อ่าน state ปัจจุบันใน async / side-effect
- set(..., true) → replace ทั้ง state (ไม่ merge)
- ปกติแนะนำ partial merge (default)
- Action เป็น plain function — test ได้ตรง ๆ

```
javascript
export const useCartStore = create((set, get) => ({
  items: [],
  add: (p) =>
    set((s) => ({ items: [...s.items, p] })),
  remove: (id) =>
    set((s) => ({
      items: s.items.filter(i => i.id !== id),
    })),
  total: () =>
    get().items.reduce((sum, i) => sum + i.price, 0),
  // อ่าน state ปัจจุบันก่อนตัดสินใจ
  checkout: () => {
    if (get().items.length === 0) return;
    // ... call API
    set({ items: [] });
  },
}));
```

⚠ Pitfalls / ข้อควรระวัง

- อย่า mutate ของเดิม (เช่น s.items.push) — Zustand ใช้ shallow compare; ต้อง return ของใหม่
- total() เป็น method ไม่ใช่ derived state — re-compute ทุกครั้ง; ถ้านักให้ใช้ memo ผัง component

Selectors ลด re-render

subscribe เฉพาะ slice ที่ต้องการ

- `useStore()` แบบไม่มี selector → re-render เมื่อ state ใด ๆ เปลี่ยน
- `useStore(s => s.count)` → re-render เฉพาะตอน count เปลี่ยน
- Zustand เปรียบเทียบด้วย `Object.is` (strict equality)
- เลือก primitive ตรง ๆ ดีที่สุด
- ถ้าเลือก object/array → ต้องใช้ shallow equal

```
import { useShallow } from "zustand/react/shallow";

// ❌ ดึง object ทั้งก้อน – re-render บ่อย
const { count, inc } = useCounterStore();

// ✅ เลือกค่าทีละตัว
const count = useCounterStore(s => s.count);
const inc = useCounterStore(s => s.inc);

// ✅ เลือกหลายค่าพร้อม shallow compare
const { count, inc } = useCounterStore(
  useShallow(s => ({ count: s.count, inc: s.inc }))
);
```

⚠ Pitfalls / ข้อควรระวัง

- อย่าสร้าง object ใหม่ใน selector โดยไม่มี shallow compare — ทุก render จะ !== ของเดิม

Pattern: per-value vs whole-object

เลือกแบบไหนเมื่อไหร่?

Per-value selector

- ข้อดี: re-render น้อย, อ่านง่าย
- ข้อเสีย: เขียนหลายบรรทัดถ้าใช้หลายค่า
- ใช้กับ: primitive, action ที่เรียกบ่อย

```
const a = useStore(s => s.a);
const b = useStore(s => s.b);
const c = useStore(s => s.c);
```

Whole-object + useShallow

- ข้อดี: เขียนรวบรวบบรรทัดเดียว
- ข้อเสีย: ลืม useShallow → bug performance
- ใช้กับ: หลายค่า, ดึง action group

```
const { a, b, c } = useStore(
  useShallow(s => ({
    a: s.a, b: s.b, c: s.c
  })))
);
```

⚠ Pitfalls / ข้อควรระวัง

- Default rule: เริ่มด้วย per-value ก่อน — ใช้ shallow ก็ต่อเมื่อยุ่งยากจริง

โครงสร้างโฟลเดอร์ src/stores/

แยก domain – ไม่ยัดทุกอย่างใน global store

- 1 store = 1 domain (auth, cart, ui, todos...)
- ไม่ทำ god store ก้อนเดียว — ดูแลยาก
- vanilla = createStore แยกออกถ้าใช้นอก React
- types/ เก็บ TS interface ของ state
- selectors/ (optional) แยก derived selector ที่ใช้ซ้ำ

```
src/  
├── stores/  
│   ├── index.ts           // barrel export  
│   ├── auth.store.ts  
│   ├── cart.store.ts  
│   ├── ui.store.ts  
│   └── types.ts  
├── services/              // API calls  
├── components/  
└── pages/  
  
// stores/index.ts  
export * from "./auth.store";  
export * from "./cart.store";  
export * from "./ui.store";
```

⚠ Pitfalls / ข้อควรระวัง

- ไม่ต้องเอา local UI state (เช่น isOpen ของ modal เดี่ยว) เข้า store — ใช้ useState ในที่ของมัน

Pitfalls — React patterns

ข้อผิดพลาดที่เจอบ่อยและวิธีแก้

- 1 Select object ทั้งก่อน → ใช้ per-value หรือ useShallow
- 2 สร้าง selector inline ที่ return object ใหม่ → ดึง primitive ที่ละตัว
- 3 เก็บ derived state (เช่น total) ใน store → คำนวณตอนใช้ผ่าน selector / memo
- 4 ใช้ store เป็น 'global variable' ของทุกอย่าง → แยก domain, useState สำหรับ local
- 5 Mutate array/object เดิมใน set → return reference ใหม่เสมอ
- 6 สืมใช้ get() ใน async action → ค่า stale; ใช้ get() อ่านล่าสุด

Async Actions

fetch / API call ภายใน store ได้เลย

- Action เป็น function ปกติ → ใช้ async/await ได้
- track loading + error เป็น state ใน store
- set ก่อน fetch → set หลังได้ผล
- ใช้ get() อ่าน state ปัจจุบันก่อนตัดสินใจ
- ป้องกัน race: ใช้ AbortController / request id



javascript

```
export const useUserStore = create((set, get) => ({
  user: null,
  loading: false,
  error: null,

  fetchUser: async (id) => {
    if (get().loading) return;
    set({ loading: true, error: null });
    try {
      const res = await fetch(`/api/users/${id}`);
      if (!res.ok) throw new Error(res.statusText);
      const user = await res.json();
      set({ user, loading: false });
    } catch (e) {
      set({ error: e.message, loading: false });
    }
  },
}));
```

Service Layer + Loading/Error

แยก fetch logic ออกจาก store

- Store ไม่ควรรู้รายละเอียด HTTP / endpoint
- สร้าง services/userService.ts รับผิดชอบ fetch
- Store เรียก service แล้ว map ผลเข้า state
- Test store ง่ายขึ้น — mock service ได้
- AbortController กัน race; ทำ retry/timeout ตรงนี้

```
javascript

// services/userService.ts
export async function getUser(id, signal) {
  const r = await fetch(`/api/users/${id}`, { signal });
  if (!r.ok) throw new Error(r.statusText);
  return r.json();
}

// stores/user.store.ts
import { getUser } from "@services/userService";

fetchUser: async (id) => {
  get().controller?.abort();
  const c = new AbortController();
  set({ loading: true, controller: c });
  try {
    const user = await getUser(id, c.signal);
    set({ user, loading: false });
  } catch (e) {
```

⚠ Pitfalls / ข้อควรระวัง

- อย่าโยน Response object หรือ fetch error ดิบลง store — sanitize ให้เหลือแค่ข้อมูลที่ใช้

Vanilla store: ใช้นอก React

createStore – ทำให้ Zustand ใช้ได้ทุกที่

- `import { createStore } from 'zustand/vanilla'`
- ได้ object ที่มี: `getState`, `setState`, `subscribe`, `getInitialState`
- ไม่มี hook — ใช้ใน `node script`, `worker`, `test` ได้
- ใน React: ห่อด้วย `useStore(store, selector)`
- Pattern นี้ดีสำหรับ `websocket` / `background sync`

```
import { createStore } from "zustand/vanilla";
import { useStore } from "zustand";

export const counterStore = createStore((set) => ({
  count: 0,
  inc: () => set((s) => ({ count: s.count + 1 })),
}));

// outside React
counterStore.getState().inc();
counterStore.subscribe(
  (s) => console.log("count =", s.count)
);

// inside React
const useCounter = (sel) => useStore(counterStore, sel);
const count = useCounter((s) => s.count);
```

Use cases — ใช้นอก React

WebSocket, background sync, integration test

WebSocket sync

- ▶ รับ message → setState
- ▶ Component subscribe ได้ทันที
- ▶ ตัด WS แยกจาก UI lifecycle

Background sync

- ▶ sync state ↔ IndexedDB
- ▶ ทำใน worker/timer
- ▶ ไม่ต้อง mount component

Integration test

- ▶ import store → setState mock
- ▶ assert ผลผ่าน getState
- ▶ test action โดยไม่ต้องมี React

⚠ Pitfalls / ข้อควรระวัง

- ระวัง memory leak: ทุก subscribe ค้นฟังก์ชัน unsubscribe — เรียกตอน cleanup เสมอ

03

Middleware

`persist · devtools · immer ·
subscribeWithSelector`

persist 101

เก็บ state ซ้ำม reload ด้วย localStorage

- ห่อ initializer ด้วย persist(initializer, options)
- options.name → key ใน storage (required)
- options.storage → createJSONStorage(() => localStorage)
- Zustand sync state ↔ storage อัตโนมัติ
- รองรับ async storage (AsyncStorage, IndexedDB) ผ่าน adapter

```
import { create } from "zustand";
import {
  persist, createJSONStorage
} from "zustand/middleware";

export const usePrefsStore = create(
  persist(
    (set) => ({
      theme: "light",
      setTheme: (theme) => set({ theme }),
    }),
    {
      name: "prefs", // storage key
      storage: createJSONStorage(
        () => localStorage
      ),
      partialize: (s) => ({ theme: s.theme }),
      version: 1,
    }
  )
);
```

persist options ละเอียด

name · partialize · version · migrate · merge · skipHydration

name	string key ใน storage (required)
storage	createJSONStorage(() => localStorage sessionStorage AsyncStorage)
partialize	(s) => slice — เก็บเฉพาะ field ที่ต้องการ
version	number — ถ้า schema เปลี่ยนต้องเพิ่มเลข
migrate	(persisted, version) => newState — แปลงข้อมูลเก่า
merge	(persisted, current) => merged — deep merge logic เอง
skipHydration	true → ต้องเรียก useStore.persist.rehydrate() เอง
onRehydrateStorage	() => (state, error) => {...} — hook หลัง rehydrate

⚠ Pitfalls / ข้อควรระวัง

- ห้าม persist token / password ลง localStorage แบบ plain — ใช้ httpOnly cookie หรือเข้ารหัส
- ลืมเพิ่ม version + migrate → user เก่าโหลดเจอ schema mismatch แล้วแอป crash

devtools middleware

Redux DevTools + time travel

- ห่อด้วย devtools(initializer, { name })
- ติดตั้ง Redux DevTools extension ในเบราว์เซอร์
- เห็นทุก action + diff state + time travel
- ใส่ชื่อ action ใน set: set(fn, false, 'auth/login')
- เปิดเฉพาะ dev environment — ปิดใน production

```
javascript
import { create } from "zustand";
import { devtools } from "zustand/middleware";

export const useAuthStore = create(
  devtools(
    (set) => ({
      user: null,
      login: (u) =>
        set({ user: u }, false, "auth/login"),
      logout: () =>
        set({ user: null }, false, "auth/logout"),
    }),
    {
      name: "auth-store",
      enabled: import.meta.env.DEV, // dev only
    }
  )
);
```

⚠ Pitfalls / ข้อควรระวัง

- อย่าเปิด devtools ใน production — leak state ออกผ่าน extension และเพิ่ม overhead

immer & subscribeWithSelector

เลือกใช้เมื่อโครงสร้าง state ซ้อนลึก / ต้องการ subscribe นอก React

immer

เขียน mutation ตรง ๆ; immer ทำ immutable ให้

```
import { immer } from
  "zustand/middleware/immer";

create(immer((set) => ({
  todos: [],
  add: (t) => set((s) => {
    s.todos.push(t); // OK!
  }),
})))
```

subscribeWithSelector

subscribe เฉพาะ slice นอก React

```
import { subscribeWithSelector }
  from "zustand/middleware";

const useStore = create(
  subscribeWithSelector((set) => ({...}))
);
useStore.subscribe(
  (s) => s.count,
  (count) => console.log(count)
);
```

⚠ Pitfalls / ข้อควรระวัง

- อย่าใช้ immer ทุก store เป็น default — เพิ่ม bundle และ overhead เล็กน้อย; ใช้เมื่อ nested state ลึกจริง

SSR / Next.js — Hydration Pitfalls

`persist + localStorage` ↔ `server rendering` ไม่ตรงกัน

- Server ไม่มี `localStorage` → render ด้วยค่า default
- Client mount → rehydrate จาก storage → state เปลี่ยน
- ผล: HTML server ≠ client → hydration mismatch warning
- แก้ด้วย `skipHydration` + เรียก `rehydrate()` ฝั่ง client
- หรือใช้ `useEffect` + state flag กัน render ก่อน hydrate

```
javascript

// 1) skipHydration: ปิด auto-rehydrate
persist(init, {
  name: "prefs",
  skipHydration: true,
});

// 2) เรียก rehydrate ฝั่ง client เท่านั้น
"use client";
useEffect(() => {
  usePrefsStore.persist.rehydrate();
}, []);

// 3) กัน flicker ระหว่างรอ rehydrate
const hydrated = usePrefsStore(
  (s) => s._hasHydrated
);
```

⚠ Pitfalls / ข้อควรระวัง

- ห้ามแชร์ store เดียวข้าม request บน server — leak ข้อมูลของ user คนอื่น
- ใน Next.js App Router → สร้าง store ใน client component หรือผ่าน Context provider per-request

Testing Strategy

unit test store, mock service, async actions

- Action เป็น pure function → test ง่ายที่สุด
- import store → call action → assert getState()
- reset state ใน beforeEach (เก็บ initial state)
- Mock service layer สำหรับ async action
- ทดสอบ middleware (persist) ด้วย mock storage

⚠ Pitfalls / ข้อควรระวัง

- อย่าลืม reset state ทุก test — Zustand store เป็น singleton ต่อ module

```
javascript

import { act } from "@testing-library/react";
import { useCounterStore } from "../counter.store";

const initial = useCounterStore.getState();

beforeEach(() => {
  useCounterStore.setState(initial, true);
});

test("inc เพิ่มขึ้น 1", () => {
  act(() => useCounterStore.getState().inc());
  expect(useCounterStore.getState().count).toBe(1);
});

test("fetchUser sets user", async () => {
  jest.spyOn(svc, "getUser").mockResolvedValue({id:1});
  await useUserStore.getState().fetchUser(1);
});
```

04

Workshop

Mini App: Todo + Filter + Theme + Auth Mock

Step 1 — Setup project

Vite + React + Zustand

- สร้างโปรเจกต์ Vite React
- ติดตั้ง zustand
- สร้างโฟลเดอร์ src/stores
- สร้าง src/services/auth.ts (mock)

```
bash

# create project
npm create vite@latest zustand-mini -- \
  --template react

cd zustand-mini
npm install
npm install zustand

# โฟลเดอร์
mkdir -p src/stores src/services
touch src/stores/todos.store.js
touch src/stores/prefs.store.js
touch src/stores/auth.store.js
touch src/services/auth.js

npm run dev
```

Step 2 — Todo store

เพิ่ม / ลบ / toggle todos

javascript

```
// src/stores/todos.store.js
import { create } from "zustand";

let nextId = 1;

export const useTodosStore = create((set, get) => ({
  todos: [], // { id, text, done }

  add: (text) => set((s) => ({
    todos: [...s.todos, { id: nextId++, text, done: false }]
  })),

  toggle: (id) => set((s) => ({
    todos: s.todos.map(t =>
      t.id === id ? { ...t, done: !t.done } : t
    )
  })),

  remove: (id) => set((s) => ({
    todos: s.todos.filter(t => t.id !== id)
  })),

  clearDone: () => set((s) => ({
    todos: s.todos.filter(t => !t.done)
  })),
}));
```

Zustand v4.1.0 → 1 → Production

- 1 store ต่อ 1 domain — todos แยก
- แต่ละ action return reference ใหม่ (immutable)
- id สร้างใน module scope (mock, ไม่ persist)
- ใน production ใช้ uuid / id จาก backend
- Component: useTodosStore(s => s.todos)

Step 3 — Filter (all / active / done)

Derived state ผ่าน selector

javascript

```
// เพิ่มใน todos.store.js
filter: "all", // 'all' | 'active' | 'done'
setFilter: (f) => set({ filter: f }),

// selector แบบ derived (ไม่เก็บใน state)
export const selectVisibleTodos = (s) => {
  switch (s.filter) {
    case "active": return s.todos.filter(t => !t.done);
    case "done":   return s.todos.filter(t => t.done);
    default:      return s.todos;
  }
};

// ใน component: ใช้ useShallow ป้องกัน reference ใหม่
import { useShallow } from "zustand/react/shallow";

const todos = useTodosStore(useShallow(selectVisibleTodos));
const filter = useTodosStore(s => s.filter);
const setFilter = useTodosStore(s => s.setFilter);
```

- Filter state เก็บใน store เดียวกับ todos
- Derived list คำนวณใน selector — ไม่ persist
- selector return array ใหม่ → ห่อด้วย useShallow
- ทำให้ list re-render เฉพาะตอนข้อมูลเปลี่ยนจริง

Step 4 — Theme + persist

เก็บ theme เข้า reload

```
javascript

// src/stores/prefs.store.js
import { create } from "zustand";
import { persist, createJSONStorage } from "zustand/middleware";

export const usePrefsStore = create(
  persist(
    (set) => ({
      theme: "light", // 'light' | 'dark'
      setTheme: (theme) => set({ theme }),
      toggleTheme: () => set((s) => ({
        theme: s.theme === "light" ? "dark" : "light"
      })),
    }),
    {
      name: "prefs",
      storage: createJSONStorage(() => localStorage),
      partialize: (s) => ({ theme: s.theme }),
      version: 1,
    }
  )
);
```

- ใช้ persist + localStorage
- partialize เก็บเฉพาะ theme
- version: 1 → เพื่อ migrate ในอนาคต
- Component: ใส่ class 'dark' บน <html>
- ทดสอบ: refresh แล้ว theme คงอยู่

Step 5 — Auth mock + service layer

Async action + loading/error

```
javascript

// src/services/auth.js
export async function login(email, password) {
  await new Promise(r => setTimeout(r, 600));
  if (email === "demo@x.io" && password === "1234")
    return { id: 1, name: "Demo User", email };
  throw new Error("Invalid credentials");
}

// src/stores/auth.store.js
import { create } from "zustand";
import * as authSvc from "@services/auth";

export const useAuthStore = create((set, get) => ({
  user: null, loading: false, error: null,
  login: async (email, password) => {
    if (get().loading) return;
    set({ loading: true, error: null });
    try {
      const user = await authSvc.login(email, password);
      set({ user, loading: false });
    } catch (e) {
      set({ error: e.message, loading: false });
    }
  },
  logout: () => set({ user: null }),
})));
```

- Service layer แยก fetch logic
- Store ไม่รู้รายละเอียด API
- track loading + error แบบ explicit
- ป้องกัน double-submit ด้วย get().loading
- Test ง่าย: mock authSvc.login

Step 6 — Selectors + devtools

Performance + debugging

```
javascript

// แก้ auth.store.js เพิ่ม devtools
import { devtools } from "zustand/middleware";

export const useAuthStore = create(
  devtools(
    (set, get) => ({ /* ...เหมือนเดิม... */
      login: async (email, password) => {
        set({ loading: true }, false, "auth/loginStart");
        try {
          const user = await authSvc.login(email, password);
          set({ user, loading: false }, false, "auth/loginOk");
        } catch (e) {
          set({ error: e.message, loading: false },
            false, "auth/loginErr");
        }
      },
    }),
    { name: "auth", enabled: import.meta.env.DEV }
  )
);

// Component: ดึงทีละค่า
const user = useAuthStore(s => s.user);
const loading = useAuthStore(s => s.loading);
const login = useAuthStore(s => s.login);
```

- เปิด Redux DevTools ในเบราว์เซอร์
- เห็นทุก action: loginStart / Ok / Err
- Time travel ย้อน state ได้
- ใช้ per-value selector ทุกที่
- เปิดเฉพาะ DEV ผ่าน enabled flag

Challenge + เฉลย

ลองทำเองก่อนดูเฉลย

🌸 Challenge

- เพิ่ม per-user todos: ผู้ก todos กับ user ที่ login
- Logout แล้ว todos ของ user นั้นต้องไม่ถูกเคลียร์
- Login กลับมาเห็น todos เดิม
- persist todos ลง localStorage แยก per user
- เพิ่ม version + migrate (เพื่อ schema เปลี่ยน)

⚠️ Pitfalls / ข้อควรระวัง

- ระวัง: localStorage มี quota ~5MB; ถ้าผู้ใช้มี todos จำนวนมาก ให้ย้ายไป IndexedDB



javascript

```
// แนวเฉลย - partialize + key dynamic
persist(
  (set, get) => ({
    byUser: {}, // { [userId]: Todo[] }
    add: (uid, text) => set((s) => ({
      byUser: {
        ...s.byUser,
        [uid]: [
          ...(s.byUser[uid] ?? []),
          { id: nextId++, text, done: false },
        ],
      },
    })),
    // selector: list ของ user ปัจจุบัน
  }),
  {
    name: "todos-by-user",
    version: 1,
    migrate: (persisted, v) => {
```

Production Checklist

ก่อน merge → ก่อน release

Design

- ✓ แยก store ตาม domain
- ✓ ไม่เก็บ derived state
- ✓ ไม่ยัด local UI state เข้า global

Performance

- ✓ Selector แบบ per-value หรือ useShallow
- ✓ ไม่สร้าง object ใหม่ใน selector
- ✓ หลีกเลี่ยง heavy compute ใน selector

Persistence

- ✓ partialize เลือก field ชัดเจน
- ✓ version + migrate พร้อม
- ✓ ไม่ persist token / password ดิบ

Debugging / SSR / Test

- ✓ devtools เปิดเฉพาะ DEV
- ✓ skipHydration + per-request store ใน Next.js
- ✓ Unit test action + reset state ทุก test

Q & A

ถามได้ทุกเรื่องเกี่ยวกับ Zustand · React · State Management

Resources

github.com/pmndrs/zustand

zustand.docs.pmnd.rs

[Redux DevTools Extension](#)

Thank you! 